1 Автоматная грамматика

Спроектировать грамматику по заданному регулярному выражению и построить эквивалентный конечный автомат.

pattern = (\W|^)po[#\-]{0,1}\s{0,1}\d{2}[\s-]{0,1}\d{4}(\W|$)

Обозначения: W = \W, H = \s, D = \d, C = {#, -}, B = H ∪ {-}. ε — пустая цепочка. Якоря ^ и $ не читают символ. Рассматривается язык полных совпадений.

$L = \{\, l{\cdot}po{\cdot}c{\cdot}h{\cdot}u{\cdot}b{\cdot}v{\cdot}r \mid l, r \in \{\varepsilon\} \cup W,$

$c \in \{\varepsilon, \#, -\},\ h \in \{\varepsilon\} \cup H,\ u \in D^2,\ b \in \{\varepsilon\} \cup B,\ v \in D^4 \,\}.$

Язык регулярный: задан регулярным выражением. По теореме Клини регулярные выражения, праволинейные грамматики и конечные автоматы задают один класс языков.

$G = (\Sigma, V, P, S),$

$V = \{S, A, P, O, J, T, D_1, D_2, K, D_3, D_4, D_5, D_6\}.$

Σ — конечный входной алфавит; V — нетерминалы; P — правила вывода; S — начальный символ. Строчные w, h, d обозначают один терминал соответствующего класса W, H, D; запись задаёт семейство правил.

$p_1\colon S \to pP \mid wA\,; \quad p_2\colon A \to pP$

$p_3\colon P \to oO$

$p_4\colon O \to \#J \mid\ -\ J \mid hT \mid dD_1$

$p_5\colon J \to hT \mid dD_1\,; \quad p_6\colon T \to dD_1$

$p_7\colon D_1 \to dD_2$

$p_8\colon D_2 \to hK \mid\ -\ K \mid dD_3$

$p_9\colon K \to dD_3\,; \quad p_{10}\colon D_3 \to dD_4$

$p_{11}\colon D_4 \to dD_5\,; \quad p_{12}\colon D_5 \to dD_6 \mid d$

$p_{13}\colon D_6 \to w$

Все правила имеют вид X → aY или X → a, где X,Y ∈ V, a ∈ Σ. Это праволинейная регулярная грамматика типа 3 по Хомскому. Единичных и ε-правил нет; грамматика неукорачивающая.

Пример вывода po#□12-3456 (□ — пробел):

$S \Rightarrow pP \Rightarrow poO \Rightarrow po\#J \Rightarrow po\#\square T \Rightarrow po\#\square 1D_1$

$\Rightarrow po\#\square 12D_2 \Rightarrow po\#\square 12\ -\ K \Rightarrow po\#\square 12\ -\ 3D_3$

$\Rightarrow po\#\square 12\ -\ 34D_4 \Rightarrow po\#\square 12\ -\ 345D_5 \Rightarrow po\#\square 12\ -\ 3456.$

Минимальная длина — 8 символов, максимальная — 13. Например, po123456 принимается, а po12345 и po##123456 отвергаются.

2 Построение конечного автомата

Конечный автомат — модель с конечной памятью состояний, читающая вход посимвольно. ДКА задаётся пятёркой (Q,Σ,δ,q_0,F), где Q — состояния, Σ — алфавит, $\delta: Q \times \Sigma \to Q$ — функция переходов, q_0 — начальное состояние, $F \subseteq Q$ — заключительные состояния.

В ε-НДКА функция δ отображает пару из $Q \times (\Sigma \cup \{\varepsilon\})$ в подмножество Q и допускает несколько продолжений и переходы без чтения символа. Язык принят, если хотя бы один путь читает всю цепочку и заканчивается в F.

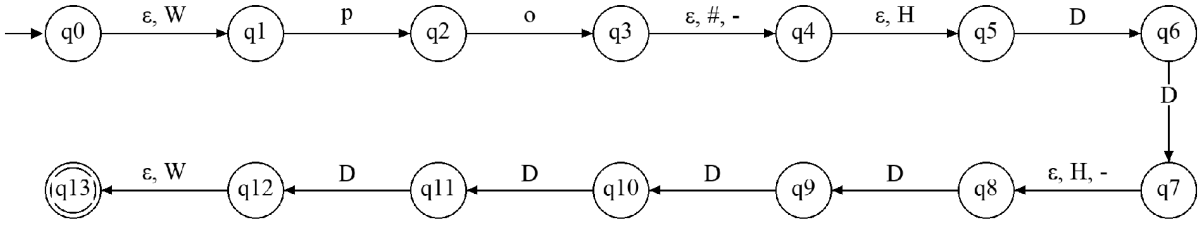
Построение Томпсона и компактный ε-НДКА

Запишем выражение как W?·p·o·C?·H?·D·D·B?·D·D·D·D·W?. Для атома a строятся вход u и выход v с дугой u —a→ v. Для X? добавляются общий вход и выход с ε-обходом X; при конкатенации конец левого фрагмента соединяется с началом правого ε-дугой.

При двух состояниях на каждый из 13 атомов и двух на каждую из пяти оболочек ? получается 36 состояний. Здесь C и B — символьные классы, каждый считается одним атомом.

Склеим ε-соединения конкатенации. Необязательный атом оформим двумя параллельными дугами между его входом и выходом: по ε и по классу. Это даёт эквивалентный компактный ε-НДКА из 14 состояний. Именно его детерминизация приведена далее.

$M_N = (Q_N, \Sigma, \delta N, q_0, \{q_{13}\}), \quad Q_N = \{q_0, \ldots, q_{13}\}.$



Метки на диаграмме перечисляют альтернативные переходы. Неуказанные переходы равны ∅. q_0 — начальное состояние, q_{13} — единственное заключительное. Полученный автомат допускает тот же язык, что и G.

Алгоритм преобразования ε-НДКА в ДКА

1. Вычислить ε-замыкание начального состояния: $E(\{q_0\}) = \{q_0,q_1\}$.
2. Для подмножества X и входного класса a объединить переходы по a из всех $q \in X$, затем взять ε-замыкание результата.
3. Каждое новое подмножество добавить в очередь. Повторять до её исчерпания.
4. Подмножество заключительно, если содержит q_{13}. Пустое подмножество ∅ — состояние ошибки ERR.

Лемма о накачке

Для регулярного языка существует n: любое $w \in L$ длины $\geq n$ представимо как xyz, где $|xy| \leq n$, $|y| > 0$ и $xy^iz \in L$ для всех $i \geq 0$. Здесь $|w| \leq 13$; при конечном Σ язык конечен. Можно взять n = 14: слов такой длины в L нет.

2 Детерминизация и модель ДКА

$S = \{q_0, q_1\};\ A = \{q_1\};\ P = \{q_2\};\ O = \{q_3, q_4, q_5\};$

$J = \{q_4, q_5\};\ T = \{q_5\};\ D_1 = \{q_6\};\ D_2 = \{q_7, q_8\};$

$K = \{q_8\};\ D_3 = \{q_9\};\ D_4 = \{q_{10}\};\ D_5 = \{q_{11}\};$

$D_6 = \{q_{12}, q_{13}\};\ F = \{q_{13}\};\ ERR = \emptyset.$

$M_D = (Q_D, \Sigma, \delta D, S, \{D_6, F\}),$

$Q_D = \{S, A, P, O, J, T, D_1, D_2, K, D_3, D_4, D_5, D_6, F, ERR\}.$

Для полного разбиения алфавита используем p, o, D, H, #, -, N_0 = W \ (H ∪ {#,-}) и W_o = \w \ ({p,o} ∪ D). Классы не пересекаются. W = H ∪ {#,-} ∪ N_0; W в формулах обозначает любой символ этого объединения.

Функция переходов

$\delta D(S, p) = P;\ \delta D(S, W) = A;\ \delta D(A, p) = P;\ \delta D(P, o) = O;$

$\delta D(O, \#) = J;\ \delta D(O, -) = J;\ \delta D(O, H) = T;\ \delta D(O, D) = D_1;$

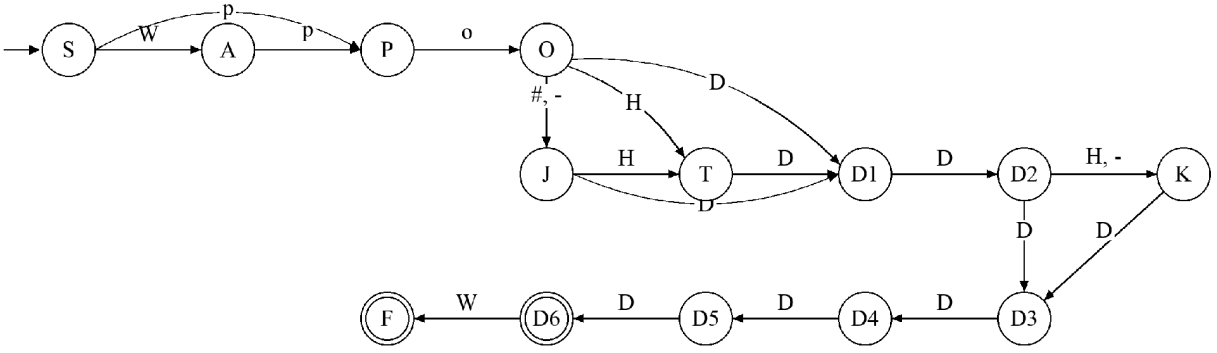
$\delta D(J, H) = T;\ \delta D(J, D) = D_1;\ \delta D(T, D) = D_1;$

$\delta D(D_1, D) = D_2;\ \delta D(D_2, H) = K;\ \delta D(D_2, -) = K;$

$\delta D(D_2, D) = D_3;\ \delta D(K, D) = D_3;\ \delta D(D_3, D) = D_4;$

$\delta D(D_4, D) = D_5;\ \delta D(D_5, D) = D_6;\ \delta D(D_6, W) = F.$

Все остальные переходы ведут в ERR; δD(ERR,a) = ERR для любого a.



Двойным кругом отмечены D_6 и F. ERR и дуги в него опущены на диаграмме; они полностью заданы правилом выше.

Примеры конфигураций

$(S, po123456) \vdash (P, o123456) \vdash (O, 123456)$

$\vdash (D_1, 23456) \vdash (D_2, 3456) \vdash (D_3, 456)$

$\vdash (D_4, 56) \vdash (D_5, 6) \vdash (D_6, \varepsilon).$

Цепочка принимается: D_6 заключительно. Для po12345 последняя конфигурация (D_5,ε); цепочка отвергается, поскольку D_5 не заключительно.

3 Минимизация ДКА

Автоматы эквивалентны, если допускают один язык. Состояния p и q неразличимы, если любое продолжение w ∈ Σ* приводит их одновременно в заключительные либо одновременно в незаключительные состояния. Минимизация объединяет неразличимые состояния достижимого ДКА.

Шаг 1. Полнота и достижимость

Автомат полный за счёт ERR. Последовательно выделим множества впервые достигнутых состояний; начинаем с $R_0 = \{S\}$.

$R_1 = \{A, P, ERR\};\ R_2 = \{O\};\ R_3 = \{J, T, D_1\};\ R_4 = \{D_2\};$

$R_5 = \{K, D_3\};\ R_6 = \{D_4\};\ R_7 = \{D_5\};\ R_8 = \{D_6\};\ R_9 = \{F\};\ R_{10} = \emptyset.$

$Reach = \cup R_i = Q_D;\quad U = Q_D \setminus Reach = \emptyset.$

Все 15 состояний достижимы. Например: A — по !, J — по po#, T — по po□, K — по po12-, D_6 — по po123456, F — по po123456!, ERR — по x.

Шаг 2. Разбиения по алгоритму Мура

Сначала разделяем заключительные и незаключительные состояния. На каждом шаге уточняем блоки: состояния остаются вместе, только если по каждому входному классу переходят в один блок предыдущего разбиения. | разделяет блоки.

$\Pi_0 = \{S, A, P, O, J, T, D_1, D_2, K, D_3, D_4, D_5, ERR\} \mid \{D_6, F\}$

$\Pi_1 = \{S, A, P, O, J, T, D_1, D_2, K, D_3, D_4, ERR\} \mid \{D_5\} \mid \{D_6\} \mid \{F\}$

$\Pi_2 = \{S, A, P, O, J, T, D_1, D_2, K, D_3, ERR\} \mid \{D_4\} \mid \{D_5\} \mid \{D_6\} \mid \{F\}$

$\Pi_3 = \{S, A, P, O, J, T, D_1, D_2, K, ERR\} \mid \{D_3\} \mid \{D_4\} \mid \{D_5\} \mid \{D_6\} \mid \{F\}$

$\Pi_4 = \{S, A, P, O, J, T, D_1, ERR\} \mid \{D_2, K\} \mid \{D_3\} \mid \{D_4\} \mid \{D_5\} \mid \{D_6\} \mid \{F\}$

$\Pi_5 = \{S, A, P, O, J, T, ERR\} \mid \{D_1\} \mid \{D_2\} \mid \{K\} \mid \{D_3\} \mid \{D_4\} \mid \{D_5\} \mid \{D_6\} \mid \{F\}$

$\Pi_6 = \{S, A, P, ERR\} \mid \{O, J, T\} \mid \{D_1\} \mid \{D_2\} \mid \{K\} \mid \{D_3\} \mid \{D_4\} \mid \{D_5\} \mid \{D_6\} \mid \{F\}$

$\Pi_7 = \{S, A, ERR\} \mid \{P\} \mid \{O\} \mid \{J\} \mid \{T\} \mid \{D_1\} \mid \{D_2\} \mid \{K\} \mid \{D_3\} \mid \{D_4\} \mid \{D_5\} \mid \{D_6\} \mid \{F\}$

$\Pi_8 = \{S, A\} \mid \{ERR\} \mid \{P\} \mid \{O\} \mid \{J\} \mid \{T\} \mid \{D_1\} \mid \{D_2\} \mid \{K\} \mid \{D_3\} \mid \{D_4\} \mid \{D_5\} \mid \{D_6\} \mid \{F\}$

$\Pi_9 = \{S\} \mid \{A\} \mid \{ERR\} \mid \{P\} \mid \{O\} \mid \{J\} \mid \{T\} \mid \{D_1\} \mid \{D_2\} \mid \{K\} \mid \{D_3\} \mid \{D_4\} \mid \{D_5\} \mid \{D_6\} \mid \{F\}$

$\Pi_{10} = \Pi_9.$

На шаге 1 D_6 и F разделяются по W, а D_5 — по D. Затем по D отделяются D_4, D_3 и блок $\{D_2,K\}$. На шаге 5 D_2 и K различаются по H, D_1 отделяется по D. Далее отделяются {O,J,T}, затем P, O, J, T. На шаге 8 {S,A} отделяется от ERR по p; на шаге 9 S и A различаются по W.

Шаг 3. Минимальный автомат

Π_9 устойчиво и состоит из 15 одноэлементных блоков. Каждому блоку сопоставляем состояние и наследуем переходы. Слияний нет; начальное состояние — S, заключительные — D_6 и F.

$M_{min} = (Q_D, \Sigma, \delta D, S, \{D_6, F\}),\quad |Q_D| = 15.$

ДКА уже минимален; функция переходов и диаграмма сохраняются.

4 Реализация функции переходов на Go

Каждому состоянию соответствует функция deltaX. Общая delta вызывает нужную функцию, а accepts выполняет переходы по всей цепочке. Для изменения δ редактируется соответствующая deltaX.

symbol в этой реализации классифицирует вход в кодировке ASCII; основной автомат задан над указанными классами.

```
package main

import (
	"fmt"
	"os"
)

// state is a state of the deterministic finite automaton.
type state uint8

const (
	stateS state = iota
	stateA
	stateP
	stateO
	stateJ
	stateT
	stateD1
	stateD2
	stateK
	stateD3
	stateD4
	stateD5
	stateD6
	stateF
	stateERR
)

// symbolClass is an ASCII input class of the automaton.
type symbolClass uint8

const (
	symbolP symbolClass = iota
	symbolO
	symbolD
	symbolH
	symbolHash
	symbolDash
	symbolN0
	symbolOther
)

// deltaS returns the transition from S.
func deltaS(a symbolClass) state {
	if a == symbolP {
		return stateP
	}

	if a == symbolH || a == symbolHash || a == symbolDash || a == symbolN0 {
		return stateA
	}

	return stateERR
}

// deltaA returns the transition from A.
func deltaA(a symbolClass) state {
	if a == symbolP {
		return stateP
	}

	return stateERR
}

// deltaP returns the transition from P.
func deltaP(a symbolClass) state {
	if a == symbolO {
		return stateO
	}

	return stateERR
```

```
}

// deltaO returns the transition from O.
func deltaO(a symbolClass) state {
	if a == symbolHash || a == symbolDash {
		return stateJ
	}

	if a == symbolH {
		return stateT
	}

	if a == symbolD {
		return stateD1
	}

	return stateERR
}

// deltaJ returns the transition from J.
func deltaJ(a symbolClass) state {
	if a == symbolH {
		return stateT
	}

	if a == symbolD {
		return stateD1
	}

	return stateERR
}

// deltaT returns the transition from T.
func deltaT(a symbolClass) state {
	if a == symbolD {
		return stateD1
	}

	return stateERR
}

// deltaD1 returns the transition from D1.
func deltaD1(a symbolClass) state {
	if a == symbolD {
		return stateD2
	}

	return stateERR
}

// deltaD2 returns the transition from D2.
func deltaD2(a symbolClass) state {
	if a == symbolH || a == symbolDash {
		return stateK
	}

	if a == symbolD {
		return stateD3
	}

	return stateERR
}

// deltaK returns the transition from K.
func deltaK(a symbolClass) state {
	if a == symbolD {
		return stateD3
	}

	return stateERR
}

// deltaD3 returns the transition from D3.
func deltaD3(a symbolClass) state {
	if a == symbolD {
		return stateD4
	}

	return stateERR
}

// deltaD4 returns the transition from D4.
```

```
func deltaD4(a symbolClass) state {
	if a == symbolD {
		return stateD5
	}

	return stateERR
}

// deltaD5 returns the transition from D5.
func deltaD5(a symbolClass) state {
	if a == symbolD {
		return stateD6
	}

	return stateERR
}

// deltaD6 returns the transition from D6.
func deltaD6(a symbolClass) state {
	if a == symbolH || a == symbolHash || a == symbolDash || a == symbolN0 {
		return stateF
	}

	return stateERR
}

// deltaF returns the transition from F.
func deltaF(_ symbolClass) state {
	return stateERR
}

// deltaERR returns the transition from ERR.
func deltaERR(_ symbolClass) state {
	return stateERR
}

// delta applies the transition function for the current state.
func delta(q state, a symbolClass) state {
	functions := []func(symbolClass) state{
		deltaS, deltaA, deltaP, deltaO, deltaJ, deltaT, deltaD1, deltaD2,
		deltaK, deltaD3, deltaD4, deltaD5, deltaD6, deltaF, deltaERR,
	}

	return functions[q](a)
}

func symbol(c byte) symbolClass {
	if c >= '0' && c <= '9' {
		return symbolD
	}

	switch c {
	case 'p':
		return symbolP
	case 'o':
		return symbolO
	case ' ', '\t', '\n', '\r', '\v', '\f':
		return symbolH
	case '#':
		return symbolHash
	case '-':
		return symbolDash
	}

	if c >= 'a' && c <= 'z' || c >= 'A' && c <= 'Z' || c == '_' {
		return symbolOther
	}

	return symbolN0
}

// accepts reports whether the automaton accepts the whole word.
func accepts(word string) bool {
	q := stateS
	for i := 0; i < len(word); i++ {
		q = delta(q, symbol(word[i]))
	}

	return q == stateD6 || q == stateF
}

// main prints one numeric acceptance result for each command-line argument.
```

```
func main() {
	for _, word := range os.Args[1:] {
		if accepts(word) {
			fmt.Println(1)
		} else {
			fmt.Println(0)
		}
	}
}
```

Компиляция: go run -v ./part-1/solution.go

Пример:

go run -v ./part-1/solution.go po123456 po#12-3456 po12345

1

1

0